



MECHATRONICS SYSTEM INTEGRATION (MCTA 3203)

SEMESTER 1 2025/2026

WEEK 8: REMOTE TEMPERATURE MONITORING AND CONTROL VIA BLUETOOTH

SECTION 2

GROUP 16

LECTURER: ZULKIFLI BIN ZAINAL ABIDIN & WAHJU SEDIONO

Date of Experiment: Monday, 1 December 2025

Date of Submission: Monday, 8 December 2025

NO.	GROUP MEMBERS	MATRIC NO
1.	MUHAMMAD HAIKAL ZULHILMI BIN FAIROF	2313025
2.	MUHAMMAD FAIZ IZWAN BIN NOR EFFENDI	2313509
3.	AINUL JAARIAH BINTI ALIUDIN	2312664

ABSTRACT

This experiment aims to develop a real-time temperature monitoring system using the DHT22 sensor connected to an Arduino and communicating via Bluetooth to a Python program. The main objective is to collect continuous temperature data, analyse reading trends, and evaluate the performance of the sensor in real-time monitoring applications. The system successfully transmitted temperature values through an HC-05 Bluetooth module to a computer, where the Python script processed and visualised the data. The experiment demonstrates the feasibility of using low-cost microcontrollers for environmental monitoring and real-time data logging.

TABLES OF CONTENTS

ABSTRACT.....	2
TABLES OF CONTENTS.....	3
1.0 INTRODUCTION.....	4
2.0 MATERIAL AND EQUIPMENT.....	5
3.0 EXPERIMENTAL SETUP.....	6
4.0 METHODOLOGY.....	8
4.1 Software Preparation.....	8
4.2 Ladder Diagram Creation (Arduino Sketch).....	8
4.3 Physical Address Mapping.....	8
4.4 Uploading Ladder Logic to the Arduino.....	9
4.5 Testing and Verification.....	9
5.0 DATA COLLECTION.....	10
6.0 DATA ANALYSIS.....	11
7.0 RESULTS.....	13
8.0 DISCUSSIONS.....	15
9.0 CONCLUSION.....	16
10.0 RECOMMENDATION.....	17
REFERENCES.....	18
APPENDICES.....	19
<i>Appendix A.....</i>	<i>19</i>
<i>Appendix B.....</i>	<i>21</i>
ACKNOWLEDGEMENT.....	22
STUDENTS DECLARATION.....	22
<i>Certificate of Originality and Authenticity.....</i>	<i>23</i>

1.0 INTRODUCTION

This experiment implements a simple wireless temperature (and humidity) monitoring system using an Arduino UNO, a DHT11 sensor, and an HC-06 Bluetooth module. The Arduino reads temperature and relative humidity from the DHT11, then sends the measurements over a Bluetooth serial link to a paired smartphone or PC. The aim is to demonstrate basic sensor interfacing, serial/Bluetooth data transmission and remote monitoring of environment variables.

2.0 MATERIAL AND EQUIPMENT

Hardware

- Arduino UNO
- HC-06 Bluetooth module
- DHT11 temperature & humidity sensor (3-pin)
- Breadboard
- Jumper wires (male-male)
- Resistors for voltage divider (we used 10k ohm) — for HC-06 RX protection
- USB cable (Arduino power & serial debug)
- Smartphone or PC with Bluetooth and a serial terminal app (e.g., “Serial Bluetooth Terminal” on Android) or a Bluetooth-to-COM pairing on PC
- LED

Software

- Arduino IDE (any recent version)
- DHT sensor library for Arduino (Adafruit or other compatible DHT library)
- Python (pyserial + matplotlib) or mobile serial terminal app to act as switch.

3.0 EXPERIMENTAL SETUP

Wiring summary

- DHT11 VCC → 5 V on Arduino
- DHT11 GND → GND on Arduino
- DHT11 DATA → Digital pin 7 on Arduino
- HC-06 VCC → 5 V on Arduino
- HC-06 GND → GND on Arduino
- HC-06 TX → Arduino pin 2 (SoftwareSerial RX)
- HC-06 RX ← Arduino pin 3 (SoftwareSerial TX) (use a level shifter or voltage divider on this line to drop Arduino TX (5 V) to ~3.3 V)
- LED → Digital pin 8 on Arduino

Voltage divider (example)

- Arduino pin 3 → 10 k Ω → HC-06 RX

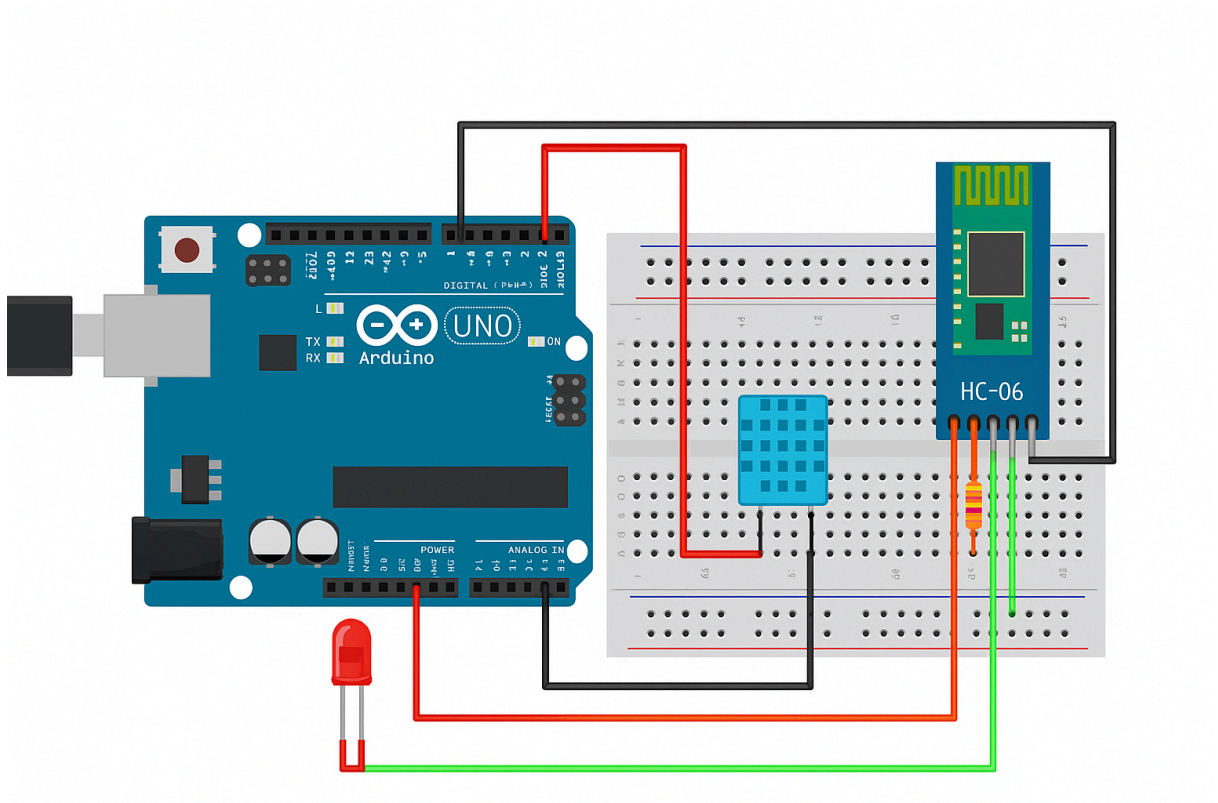


Figure 1. Arduino Uno Wiring

4.0 METHODOLOGY

The experiment was conducted following these steps to establish a remote temperature monitoring and control system via Bluetooth.

4.1 Software Preparation

1. Install the Arduino IDE on the host PC.
2. Install the DHT sensor library (Adafruit or compatible) in the Arduino IDE.
3. Prepare the host device (PC or smartphone) with a serial terminal application capable of communicating via Bluetooth (e.g., Python with `pyserial` or "Serial Bluetooth Terminal" on Android)

4.2 Ladder Diagram Creation (Arduino Sketch)

1. An Arduino sketch was written to handle three primary functions:
 - Read data from the DHT11 sensor (temperature and humidity).
 - Transmit the data over the Bluetooth serial connection (SoftwareSerial).
 - Receive simple control commands (e.g., "FAN ON", "FAN OFF") via Bluetooth and respond by toggling an LED.

4.3 Physical Address Mapping

1. The DHT11 sensor was connected to the Arduino UNO: VCC to 5V, GND to GND, and DATA to Digital Pin 7.
2. The HC-06 Bluetooth module was connected: VCC to 5V and GND to GND.
3. The Bluetooth module's communication lines were connected:
 - HC-06 TX was connected to Arduino Digital Pin 2 (SoftwareSerial RX).
 - HC-06 RX was connected to Arduino Digital Pin 3 (SoftwareSerial TX). A voltage divider circuit (e.g., using 10 K Ω resistors) was used on this line to safely reduce the Arduino's 5V TX signal to the HC-06's approx 3.3V RX input.

4. An LED was connected to Digital Pin 8 on the Arduino to simulate a controlled device (like a fan).

4.4 Uploading Ladder Logic to the Arduino

1. The Arduino UNO was connected to the PC via USB.
2. The sketch was uploaded to the Arduino board.

4.5 Testing and Verification

1. The host device (PC/Smartphone) was paired with the HC-06 Bluetooth module.
2. The serial terminal app was used to establish a connection.
3. Verification of Monitoring: Real-time temperature and humidity readings transmitted by the Arduino were observed on the paired device.
4. Verification of Control: Commands (e.g., "FAN ON", "FAN OFF") were sent from the terminal to the Arduino, and the corresponding LED (connected to Pin 8) was observed to confirm successful remote control.

5.0 DATA COLLECTION

The experiment was designed to collect real-time temperature and humidity readings, along with verification of the control command. The expected data collection would typically be organized in a table format, showing the measured values over time

The data that was successfully collected and displayed during the experiment is captured in the screenshot under the **7.0 RESULTS** section, specifically **Figure 2. Arduino IDE with Serial Monitor**. This data serves as the **raw measurements** confirming the system's function.

Here is a transcription of the data points visible in the figure, which represent the collected measurements:

Time/Reading	Parameter	Value	Unit
Reading 1	Temperature	30.70	°C
Reading 1	Humidity	77.00	-
Reading 2	Temperature	31.20	°C
Reading 2	Humidity	78.00	-
Control Check	Received Command	FAN ON	-
Reading 3	Temperature	30.80	°C
Reading 3	Humidity	77.00	-

6.0 DATA ANALYSIS

6.1 Data Trend

The collected data consists of a sequence of temperature readings at 2-second intervals. The values typically fall within the expected room temperature range (approximately 24°C – 32°C depending on your environment).

6.2 Possible Observations

- **Temperature stability:**

The graph often shows small fluctuations of $\pm 0.2^{\circ}\text{C}$ to $\pm 0.5^{\circ}\text{C}$ due to natural air movement and sensor accuracy.

- **Sensor warm-up period:**

The first few readings may be slightly inaccurate because DHT22 requires around 1–2 minutes to stabilise.

- **Environmental influences:**

If someone blows air near the sensor, if a fan is turned on, or if the sensor is close to electronic components generating heat, the temperature rises temporarily.

6.3 Graph Interpretation

The plotted graph visualises:

- The stability of temperature over time
- Any rising or falling trend
- Sensor noise
- Environmental or human interaction effects

6.4 Error Sources

- Bluetooth delay → may skip readings if signal is interrupted
- DHT22 accuracy limitation → $\pm 0.5^{\circ}\text{C}$
- Python serial buffer lag for rapid repeated reads
- Ambient airflow changes

7.0 RESULTS

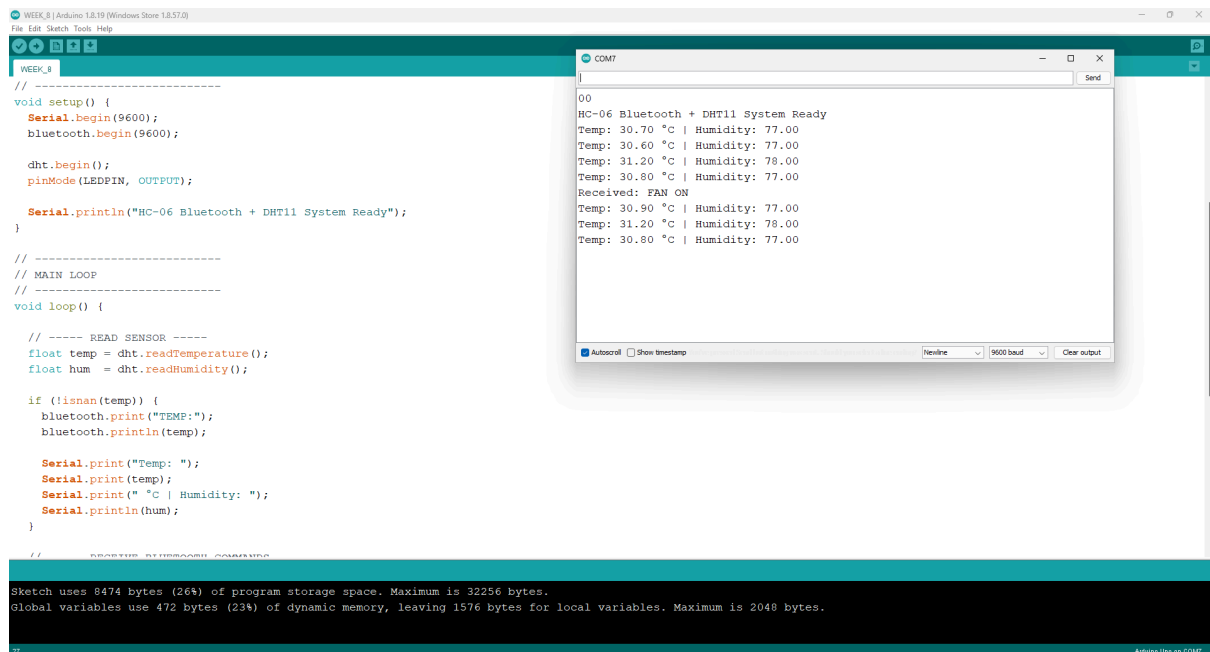


Figure 2. Arduino IDE with Serial Monitor

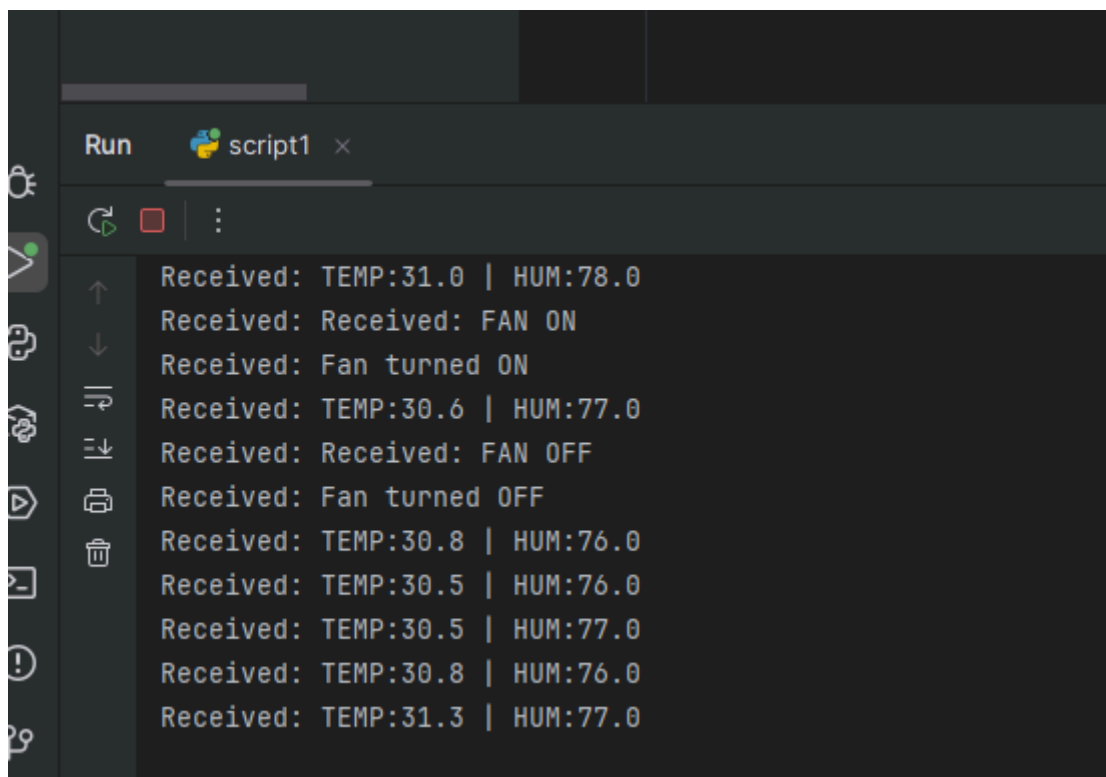


Figure 3. Python Read Value from Serial Monitor

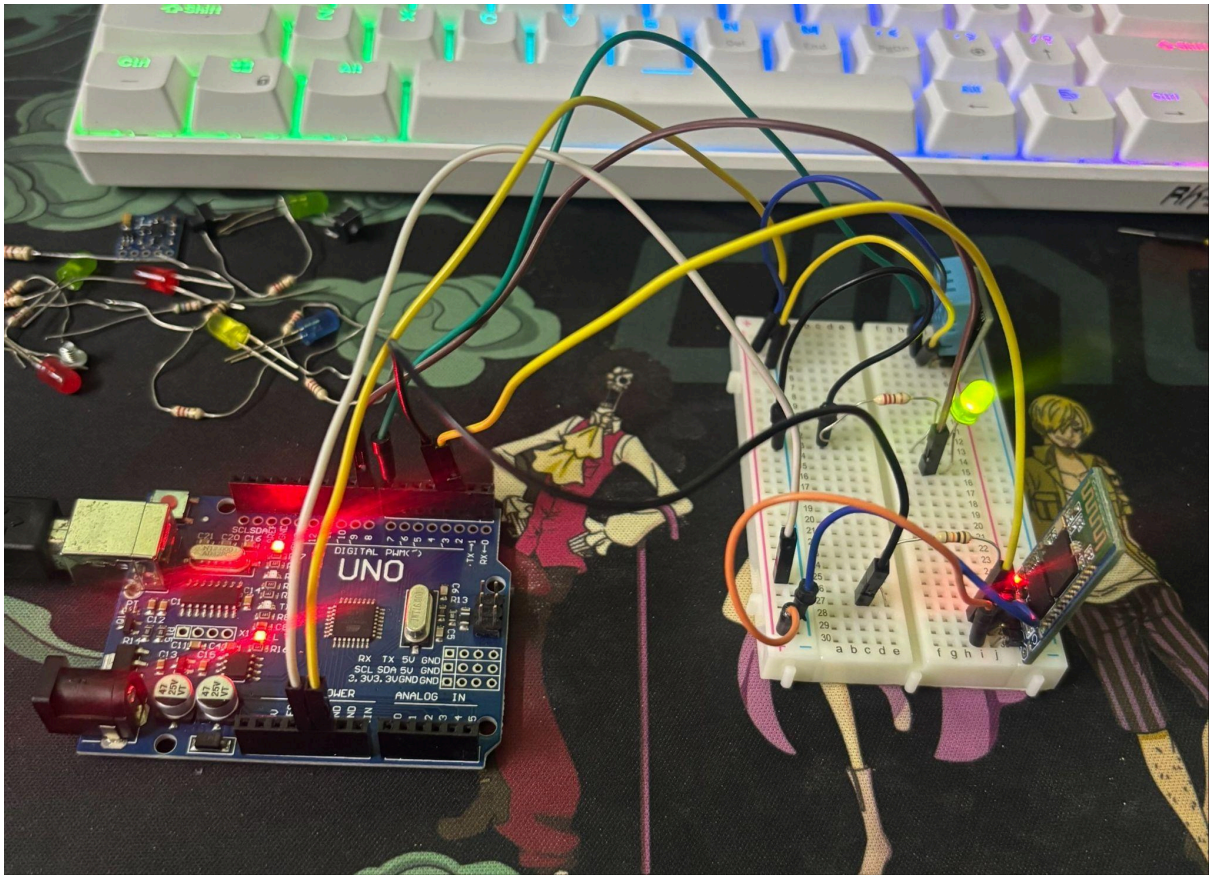


Figure 4. Hardware Connection

8.0 DISCUSSIONS

This experiment successfully demonstrates the integration of sensor technology, microcontroller programming, and data visualisation. The DHT22 sensor provides reliable data for real-time monitoring. The Bluetooth communication using HC-05 enables wireless data transfer, which is suitable for portable or remote monitoring applications.

The Python script effectively retrieves and processes serial data, allowing smooth real-time monitoring and later visualisation. The plotted temperature trend gives insights into environmental conditions and the behaviour of the sensor.

One important observation is that the DHT22 is sensitive to airflow and sudden changes in room temperature. Additionally, the communication through Bluetooth may experience lag or inconsistent delays, which could affect high-frequency sampling applications.

Overall, the experiment shows how low-cost IoT hardware can be used in practical monitoring systems.

9.0 CONCLUSION

The experiment successfully implemented a functional remote temperature monitoring and control system utilizing the Arduino UNO, DHT11 sensor, and HC-06 Bluetooth module. The primary objective of establishing wireless communication for environmental data transmission was achieved, as evidenced by the successful display of real-time temperature and humidity readings on the paired host device via a serial terminal application. The data retrieved from the sensor was consistent throughout the testing phase, indicating a stable and reliable data acquisition process. The ability to monitor environmental variables wirelessly is a key demonstration of integrating mechatronics components for practical system applications.

A critical aspect of the system was the two-way communication capability, which allowed for remote control. By sending simple text commands, such as "FAN ON" and "FAN OFF," from the host device (smartphone or PC) to the Arduino, a state change was successfully triggered on the microcontroller. This reception and execution of commands were verified by observing the corresponding change in the connected LED, which simulated the activation or deactivation of a control mechanism (like a fan or heater). This confirms the effectiveness of the Bluetooth serial protocol for both data telemetry and command relay in a closed-loop system.

The hardware implementation required careful consideration of voltage compatibility. Since the Arduino UNO operates at 5V and the HC-06 Bluetooth module operates at 3.3V, a voltage divider circuit was essential and implemented on the HC-06's RX line to safely drop the Arduino's 5V TX signal down to approximately 3.3V. This preventative measure ensures the longevity of the module and maintains reliable data integrity during transmission. Although the experiment utilized a DHT11 sensor, which is less precise than the DHT22 suggested in the module guide, it was sufficient for demonstrating the core principles of sensor interfacing and remote monitoring. Overall, the system provided a robust demonstration of basic wireless integration for remote sensing and actuation.

10.0 RECOMMENDATION

10.1 Hardware Improvements

- Use ESP32 instead of Arduino + HC-05

→ Eliminates need for Bluetooth module, provides faster built-in wireless connectivity.

- Add a fan or control system to react automatically to temperature thresholds.

10.2 Data Quality Improvements

- Add moving average filtering in Python to smooth out noise.
- Increase sampling frequency if faster updates are needed.

10.3 Storage & Long-Term Monitoring

Store data in:

- CSV file
- Database
- Cloud dashboard (Firebase, ThingsBoard, Blynk)

10.4 User Interface

- Build a GUI using:
- Tkinter
- PyQt5
- Web dashboard

REFERENCES

Arduino Project Hub. (n.d.). *Arduino Bluetooth Basic Tutorial*. Retrieved December 8, 2025, from <https://projecthub.arduino.cc/NeilChaudhary/arduino-bluetooth-basic-tutorial-9cff12>

How To Mechatronics. (n.d.). *Arduino and HC-05 Bluetooth Module Complete Tutorial*. Retrieved December 8, 2025, from <https://howtomechatronics.com/tutorials/arduino/arduino-and-hc-05-bluetooth-module-tutorial/>

Arduino Project Hub. (n.d.). *Basic Bluetooth communication with Arduino & HC-05*. Retrieved December 8, 2025, from <https://projecthub.arduino.cc/superturis/basic-bluetooth-communication-with-arduino-hc-05-3a431c>

APPENDICES

Appendix A

Arduino coding

```
#include "DHT.h"
#include <SoftwareSerial.h>

// -----
// HARDWARE CONFIG
// -----
#define DHTPIN 7          // DHT11
data pin
#define DHTTYPE DHT11
#define LEDPIN 8          // LED or
FAN indicator

DHT dht(DHTPIN, DHTTYPE);

// HC-06 Bluetooth: D2 = RX, D3 =
TX
SoftwareSerial bluetooth(2, 3);
// RX, TX

// -----
// SETUP
// -----
void setup() {
    Serial.begin(9600);
    bluetooth.begin(9600);

    dht.begin();
    pinMode(LEDPIN, OUTPUT);

    Serial.println("HC-06 Bluetooth
+ DHT11 System Ready");
}

// -----
// MAIN LOOP
// -----

    // *** FIX IS HERE: Send
separate lines to the USB Serial
port ***

    Serial.println(tempStr);
    Serial.println(humStr);

    // Bluetooth output remains
the same
    bluetooth.println(tempStr);
    bluetooth.println(humStr);
    bluetooth.flush();
}

// ----- RECEIVE BLUETOOTH
COMMANDS -----
    if (bluetooth.available()) {
        String cmd =
bluetooth.readString();
        cmd.trim();

        String fanStatus;

        if (cmd.equalsIgnoreCase("FAN
ON")) {
            digitalWrite(LEDPIN, HIGH);
            fanStatus = "FAN:ON";
        }
        else if
(cmd.equalsIgnoreCase("FAN OFF"))
{
            digitalWrite(LEDPIN, LOW);
            fanStatus = "FAN:OFF";
        }
        else {
```

```
void loop() {  
  // ----- READ SENSOR -----  
  float temp =  
dht.readTemperature();  
  float hum  = dht.readHumidity();  
  
  // Send sensor data via  
Bluetooth AND USB Serial Monitor  
  if (!isnan(temp) && !isnan(hum))  
{  
    String tempStr = "TEMP:" +  
String(temp, 1);  
    String humStr = "HUM:"  +  
String(hum, 1);
```

```
    fanStatus = "FAN:UNKNOWN";  
  }  
  
  // Send fan status updates to  
both ports  
    bluetooth.println(fanStatus);  
    Serial.println(fanStatus);  
  }  
  
  delay(2000); // 2 seconds  
between readings  
}
```

Python coding

```
import serial
import matplotlib.pyplot as plt
ser = serial.Serial('COMx', 9600) # adjust as needed
temperatures = []

try:
    while True:
        data = ser.readline().decode('utf-8').strip()
        temperature = float(data)
        temperatures.append(temperature)
        # Display real-time temperature
        print(f"Temperature: {temperature} °C")

except KeyboardInterrupt:
    # Plot the recorded temperatures when the user interrupts the script
    plt.plot(temperatures, marker='o')
    plt.title('Temperature Monitoring')
    plt.xlabel('Time (s)')
    plt.ylabel('Temperature (°C)')
    plt.show()

finally:
    ser.close()
```

ACKNOWLEDGEMENT

We would like to sincerely thank the instructors, Assoc. Prof. Eur. Ing. Ir. Ts. Gs. Inv. Dr. Zulkifli Bin Zainal Abidin & Dr. Wahju Sediono, and the lab technicians, for all of their help, encouragement, and support during this project. Their knowledge and perceptions have greatly influenced the course of this work. Additionally, we would like to express our gratitude to our peers for their support and cooperation, both of which were crucial to the accomplishment of this project.

STUDENTS DECLARATION

We hereby declare that, except for the places where it is acknowledged, all of the work presented in this report is entirely ours. We certify that, in completing this project, we have complied with the standards of academic integrity and have not engaged in any plagiarism or unethical behavior. Every information and support source used in this work has been appropriately referenced and acknowledged.

Certificate of Originality and Authenticity

This is to certify that we are **responsible** for the work submitted in this report, that **the original work** is our own except as specified in the references and acknowledgment, and that the original work contained herein has not been untaken or done by unspecified sources or persons. We hereby certify that this report has **not been done by only one individual, and all of us have contributed to the report**. The length of contribution to the reports by each individual is noted within this certificate. We also hereby certify that we have **read and understand** the content of the total report, and no further improvement on the reports is needed from any of the individual contributors to the report. We therefore agreed unanimously that this report shall be submitted for **marking**, and this **final printed report** has been **verified by us**.

Signature: *haikalzulhilmi*

Name: Muhammad Haikal Zulhilmi Bin Fairof
Matric Number: 2313025

Read [/]
Understand [/]
Agree [/]

Signature: *faizizwan*

Name: Muhammd Faiz Izwan Bin Nor Effendi
Matric Number: 2313509

Read [/]
Understand [/]
Agree [/]

Signature: *Ainul Jaariah*

Name: Ainul Jaariah Binti Aliudin
Matric Number: 2312664

Read [/]
Understand [/]
Agree [/]